



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

F.CSM101 Программчлалын үндсэн аргууд

Программчлалын хэлний тайлбар

Лекц 2

*Доктор (Ph.D) **Ганбаатарын ГАНБАТ***

2023-2024 Хавар

- Тайлбарын түвшингүүд
- Хэлзүй, синтакс
 - Контекст-Хамааралгүй хэлзүй
 - Гаргалгаа,
 - Гаргалгааны мод,
 - Утга салаалалт
- Контекст хамаарсан Синтакс шаардлага
- Компилятор
 - Лексик шинжилгээ
 - Синтакс шинжилгээ
 - Семантик шинжилгээ



ПРОГРАММЧЛАЛЫН ХЭЛ:

Алгоритмыг илэрхийлэхэд зориулан бүтээсэн формализм!

Хэдийгээр бүх зүйл нь бүтээгдмэл боловч **хэл** мөн.
Тиймээс хэл шинжлэлийн арга, хэрэгсэлд суурилан судалдаг.

Хэл шинжлэл талаасаа:



Morris, Charles William. "Foundations of the Theory of Signs." In International encyclopedia of unified science, pp. 1-59. Chicago University Press, 1938.

“Аль хэллэгүүд зөв вэ?”

1. Цагаан толгой (Alphabet)

- Тэмдэглэгээ: Анли – 22 үсэг, Кирилл – 26 үсэг, Монгол кирилл – 35 үсэг

2. Лексик компонент: **digit, number, non zero digit, 0, 1, ..., 9**

- Цагаан толгойн үгс (*words*) – *токенууд* – тэмдэглэгээний дараалал
- Түлхүүр үгс (*Keywords*), ялгагч (*identifiers*), тогтмол (*constants*), операторууд

3. Синтакс (Syntax)

- Томьёолбор хоорондын харилцан хамаарал
- Зөв хэллэг (*phrase*) – Цагаан толгой дээрх бүх үгсийн дэд олонлог

digit → 0 | 1 | ... | 9

non zero digit → 1 | ... | 9

number → non zero digit digit*

Example: C has more than 100 tokens

- Keywords, e.g., `double`, `if`, `return`, `struct`
- Identifiers, e.g., `my_var`, `printf`
- Literals, e.g., `6.022e23`, `'x'`
- Punctuators, e.g., `(`, `)`, `&&`

"Зөв хэллэг гэж юу вэ?"

- Томьёолбор болон утга агуулгын хоорондох хамаарал
 - Утга агуулга – Томьёолбороос хамаардаггүй автономит мөн чанар
- Зөв хэллэгт **онцгой** ач холбогдолтой
- *Байгалийн хэлний хувьд*: Маш комплекс, хүндрэлтэй
- *Бүтээгдмэл хэлний хувьд*: Илүү энгийн
 - Программын утга агуулга - Тооцоолж буй математик функц нь
 - Хийсвэр машин - Программын хийж буй өөрчлөлтийн мөрдлөгөө

digit → 0 | 1 | ... | 9

non zero digit → 1 | ... | 9

number → **non zero digit digit***

<- **Натурал тоо**

<- **10 хоногийн өдрүүд**

<- **Өнгөнүүд**

Time break

Common: Different syntax, same semantics

Java:

```
if (foo > 100) {  
    ...  
}
```

Bash:

```
if [ $foo -gt 100 ]  
then  
    ...  
fi
```

Sometimes: Same syntax, different semantics

Java:

Type error at
compile time

```
if ("abc" != 5) {  
    ...  
}
```

JavaScript:

Branch is
executed

“Утгатай өгүүлбэрийг хэрхэн ашиглах вэ?”

- Хэлийг хэрэглэгч буюу хүний талаас хандах ойлголт.
- Ижил утгатай өгүүлбэрийг өөр өөр байдлаар хэрэглэж болно.
- Хэл шинжлэлийн контекстууд нь өөр өөр өгүүлбэр шаарддаг;
 - зарим нь илүү ойлгомжтой,
 - зарим нь хуучирсан
 - зарим нь бусдаасаа илүү мэргэжлийн хэллэг
- Прагматик нь хэлний субъект хоорондын, нийгмийн талыг авч үздэг
 - Синтакс, семантикийг мэдэхээс чухал биш.

“Зөв өгүүлбэрийг хэрхэн гүйцэтгэх вэ?”

- Программчлалын хэлний хувьд
 - Сонгодог 3 + **Хэрэгжилтийн** түвшин
- Жишээ нь процедур хэлний хувьд
 - Зөв хэллэг нь ажиллагааг (action) тусгайлан заадаг хэл
 - “Семантикийн үүднээс зөв өгүүлбэрийг хэрхэн гүйцэтгэх”
- Хэлний хэрэглэгч семантикийн хангалттай мэдлэгтэй, Гэхдээ, ПХ-ийн дизайнер (ялангуяа хэлний дизайнер) хувьд
 - Тухайн хэллэг өөрсдийн заагдсан ажиллагааг хэрхэн биелүүлж байна вэ?
 - Хэлний хэрэгжилтээр дүрслэгдэж байж нарийн мэдэгддэг.

- Гал тогооны жорыг төлөөлөх байгалийн хэлийг авч үзье.
 - Синтакс: *Жорыг илэрхийлэх зөв өгүүлбэрүүдийг тодорхойлдог.*
 - Семантик: *Жорыг (тусгай) биелэлтээс нь үл хамааран "юу" гэдгийг тайлбарладаг.*
 - Прагматик: *Тогооч жорны өгүүлбэрүүд хэрхэн интерпретац хийж байгааг судалдаг.*
 - Хэрэгжилт : *Гал тогооны жорыг семантикийн заасан хоол болгон хувиргах аргыг (хаана, ямар найрлагатай) дүрсэлдэг.*

Хэлзүй болон Синтакс

- Хэлзүй нь синтаксийн тусламжтайгаар зөв хэлбэртэй (well-formed) өгүүлбэрүүдэд харгалзах тэмдэглэгээний дарааллыг тодорхойлдог.
- Синтаксийг хэрхэн дүрслэх вэ?
 - Байгалийн хэлний хувьд өөр байгалийн хэл. Жнь: Хэлний сурах бичиг.
 - PL-ий хувьд ихэвчлэн байгалийн хэл, ялангуяа хуучин хэлнүүдэд.
- Байгалийн хэл ихэвчлэн утга салаалдаг тул компиляц хийхэд ашиггүй.
- Чомскийн шатлал (1950-иад он)
 - Байгалийн хэлний утга салаалах байдлыг хязгаарлах.
 - Үүсгэгч хэлзүй (Generative Grammar)
 - Байгалийн хэлний синтакст ашигтай биш.
 - Программчлалын хэлний синтаксийг тайлбарлах үндсэн хэрэгсэл

Жишээ 2.1 a, b тэмдэглэгээнээс бүтсэн палиндром тэмдэг мөрийн хэл.

1. $A = \{a, b\}$ цагаан толгойг зарлана.
 2. A дээрх төгсгөлөг дарааллуудаас палиндром тэмдэгт мөрүүдийг **рекурсив** ашиглан шалгана.
 - Индукцийн үндэслэл: a, b палиндром тэмдэгт мөр.
 - Индукцийн алхам: Ямар нэг s палиндром бол asa, bsb бас мөн.
- Сондгой урттай бүх палиндромыг сонгоно
 - Тэгш урттай $abba$ гэх мэт тэмдэгт мөр?
 - Индукцийн үндэслэл дээр хоосон тэмдэгт мөрийг оруулна.

ТОД 2.1 (CFG): Контекст-хамааралгүй дүрэм нь (NT, T, R, S) дөрвөл. Энд:

1. NT – төгсгөлөг олонлог (нонтеримал, хувьсагч тэмдэглэгээ).
2. T – төгсгөлөг олонлог (терминал тэмдэглэгээ).
3. R – Гаралтын (*productions*) төгсгөлөг олонлог (*rules*), $V \rightarrow w$ хэлбэртэй. Энд V (гаралтын *толгой*) нь нэг нонтерминал тэмдэглэгээ бөгөөд w нь $T \cup NT$ дээрх тэмдэг мөр.
4. S – эхлэлийн тэмдэглээ, NT -ийн элемент

- Палиндром тэмдэгт мөрийн хэл: $(\{P\}, \{a, b\}, R, P)$

- Гаралтын дүрэм: $P \rightarrow$

$$P \rightarrow a$$

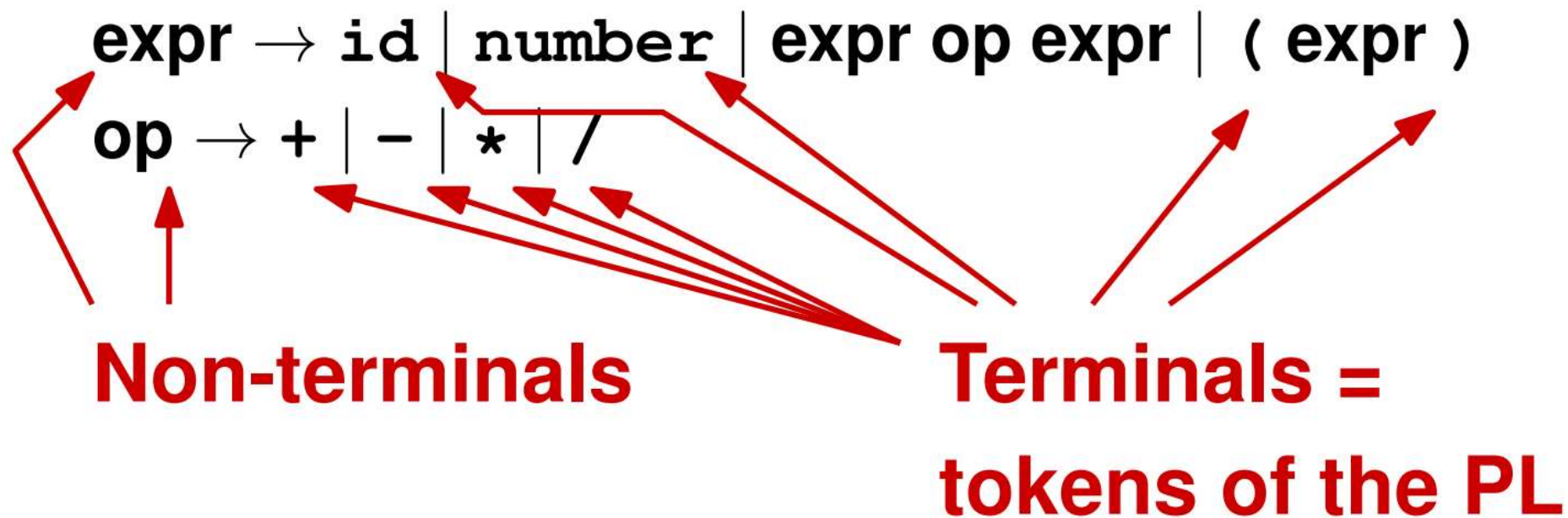
$$P \rightarrow b$$

$$P \rightarrow aPa$$

$$P \rightarrow bPb$$

≈ **Regular expressions + Recursion**

Example: Arithmetic expressions

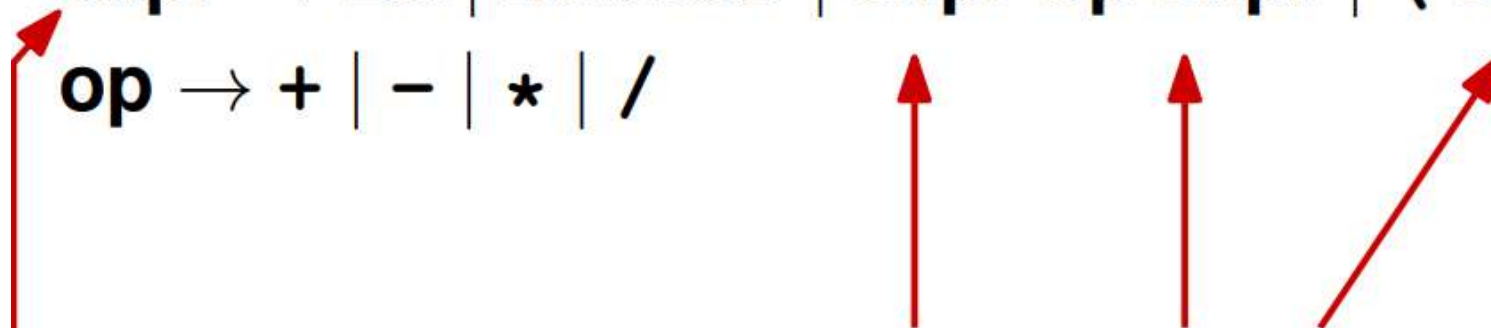


≈ **Regular expressions + Recursion**

Example: Arithmetic expressions

expr → **id** | **number** | **expr op expr** | **(expr)**

op → **+** | **-** | ***** | **/**



**Recursion allows for
nesting expressions**

Жишээ 2.1 Арифметик илэрхийллийг (+,*,−, унар, бинар) дүрслэх хялбар хэлзүйг авч үзье. Илэрхийллийн атомик элементүүд нь хялбар ялгагчид байх бөгөөд ***a*** ба ***b*** тэмдэглэгээний төгсгөлөг дараалал хэлбэртэй байна.

- $G = (\{E, I\}, \{a, b, +, *, -, (,)\}, R, E)$
 - E илэрхийлэл, I ялгагч (identifier)
- Палиндромын жишээнээс ялгаатай нь олон нонтерминалтай: E, I

$$1. E \rightarrow I$$

$$2. E \rightarrow E + E$$

$$3. E \rightarrow E * E$$

$$4. E \rightarrow E - E$$

$$5. E \rightarrow -E$$

$$6. E \rightarrow (E)$$

$$7. I \rightarrow a$$

$$8. I \rightarrow b$$

$$9. I \rightarrow Ia$$

$$10. I \rightarrow Ib$$

ТОД 2.2 (Гаргалгаа): $G = (NT, T, R, S)$ хэлзүй, $NT \cup T$ дээрх v ба w тэмдэгт мөрүүд өгөгдсөн. Хэрэв R -ийн толгой нь V байх гаралтын их биеийг v -ийн нонтерминал тэмдэглэгээнд орлуулан w -г v -ээс олж авдаг бол w -г v -ээс **шууд гаргалгаатай** эсвэл **дахин бичсэн (rewriting)** гээд $v \Rightarrow w$ гэж тэмдэглэнэ.

- $v \Rightarrow w_0 \Rightarrow w_1 \Rightarrow \dots \Rightarrow w$ гэдгийг $v \Rightarrow^* w$;
- шууд гаргалгааны төгсгөлөг үргэлжлэл
- хоосон байх боломжтой

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow E - E$
5. $E \rightarrow -E$
6. $E \rightarrow (E)$

7. $I \rightarrow a$
8. $I \rightarrow b$
9. $I \rightarrow Ia$
10. $I \rightarrow Ib$

$ab * (a + b)$

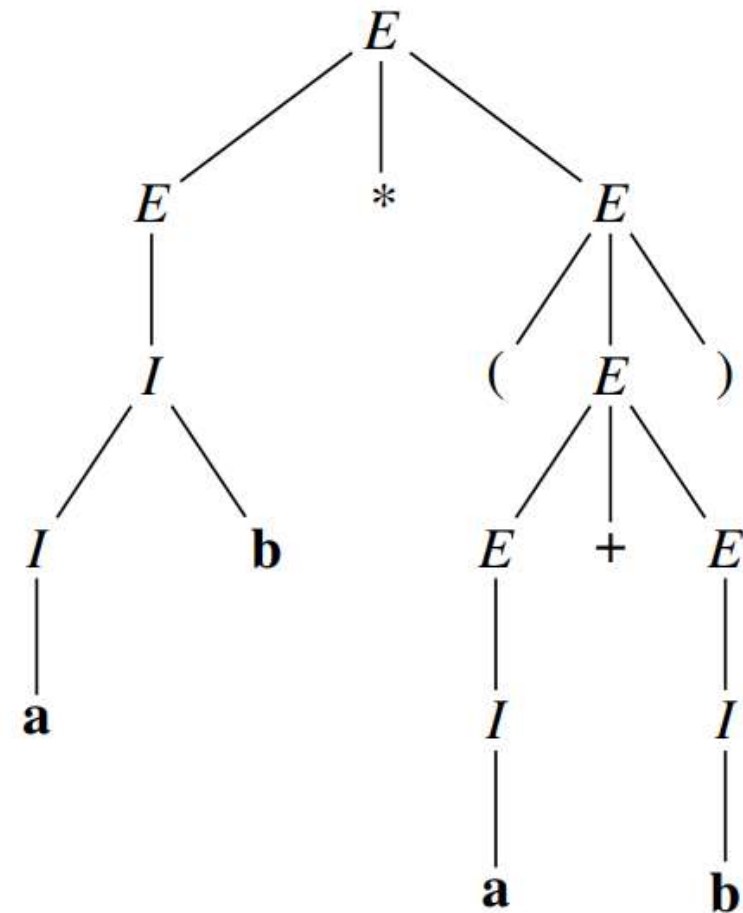
тэмдэгт мөрийн гаргалгаа

$E \Rightarrow_3 E * E$
 $\Rightarrow_1 I * E$
 $\Rightarrow_{10} Ib * E$
 $\Rightarrow_7 ab * E$
 $\Rightarrow_6 ab * (E)$
 $\Rightarrow_2 ab * (E + E)$
 $\Rightarrow_1 ab * (I + E)$
 $\Rightarrow_7 ab * (a + E)$
 $\Rightarrow_1 ab * (a + I)$
 $\Rightarrow_8 ab * (a + b)$

- Гаргалгааны/задаргааны мод нь эрэмбэтэй
- Синтаксийн шинжилгээнд чухал үүрэгтэй

Жишээ: $ab * (a + b)$ тэмдэгт мөр

$E \Rightarrow_3 E * E$
 $\Rightarrow_6 E * (E)$
 $\Rightarrow_2 E * (E + E)$
 $\Rightarrow_1 E * (E + I)$
 $\Rightarrow_8 E * (E + b)$
 $\Rightarrow_1 E * (I + b)$
 $\Rightarrow_7 E * (a + b)$
 $\Rightarrow_1 I * (a + b)$
 $\Rightarrow_{10} Ib * (a + b)$
 $\Rightarrow_7 ab * (a + b)$

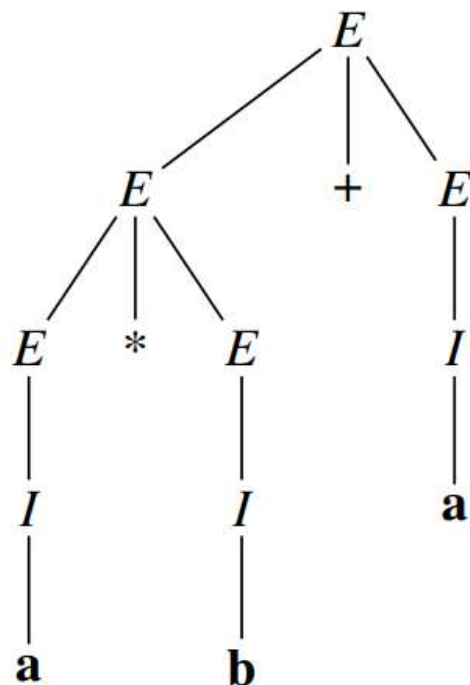


- Зүүн-баруун, баруун-зүүн уншилтын ялгаа
- Гаргалгааны олон модтой байвал утга салаалалтыг дэмждэг хэлзүй байна

Жишээ: $a * b + a$ тэмдэгт мөр

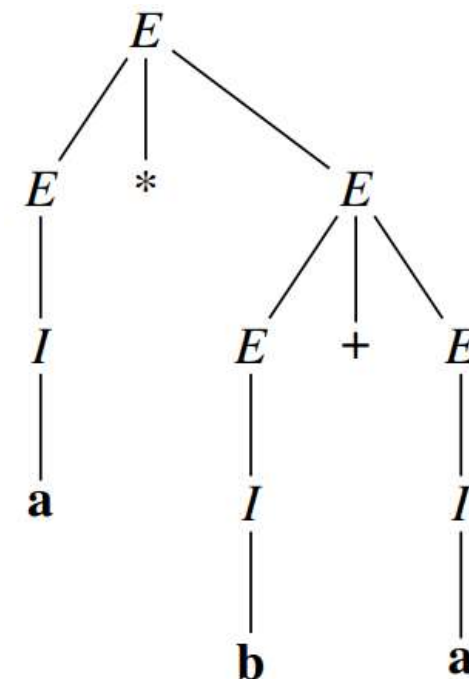
Гаргалгаа 1

$E \Rightarrow_2 E + E$
 $\Rightarrow_3 E * E + E$
 $\Rightarrow_1 I * E + E$
 $\Rightarrow_7 a * E + E$
 $\Rightarrow_1 a * I + E$
 $\Rightarrow_8 a * b + E$
 $\Rightarrow_1 a * b + I$
 $\Rightarrow_7 a * b + a$



Гаргалгаа 2

$E \Rightarrow_3 E * E$
 $\Rightarrow_1 I * E$
 $\Rightarrow_7 a * E$
 $\Rightarrow_2 a * E + E$
 $\Rightarrow_1 a * I + E$
 $\Rightarrow_8 a * b + E$
 $\Rightarrow_1 a * b + I$
 $\Rightarrow_7 a * b + a$



- Арифметик операторын ердийн эрэмбээр интерпретац хийдэг.
 - Эхлээд унар хасах ("−"), дараа нь *, тэгээд бинар + − (ижил эрэмбэтэй).

$$G' = (\{E, T, A, I\}, \{a, b, +, *, -, (,)\}, R', E)$$

$$E \rightarrow T \mid T + E \mid T - E$$

$$T \rightarrow A \mid A * T$$

$$A \rightarrow I \mid -A \mid (E)$$

$$I \rightarrow a \mid b \mid Ia \mid Ib$$

- Утга салаалахгүй ч илүү комплекс үйлдэлтэй байх шаардлага нэмэгддэг
- else аль if нөхцөлд харьяалагдах вэ?

```
if (expression1) if (expression2) command1;  
else command2;
```

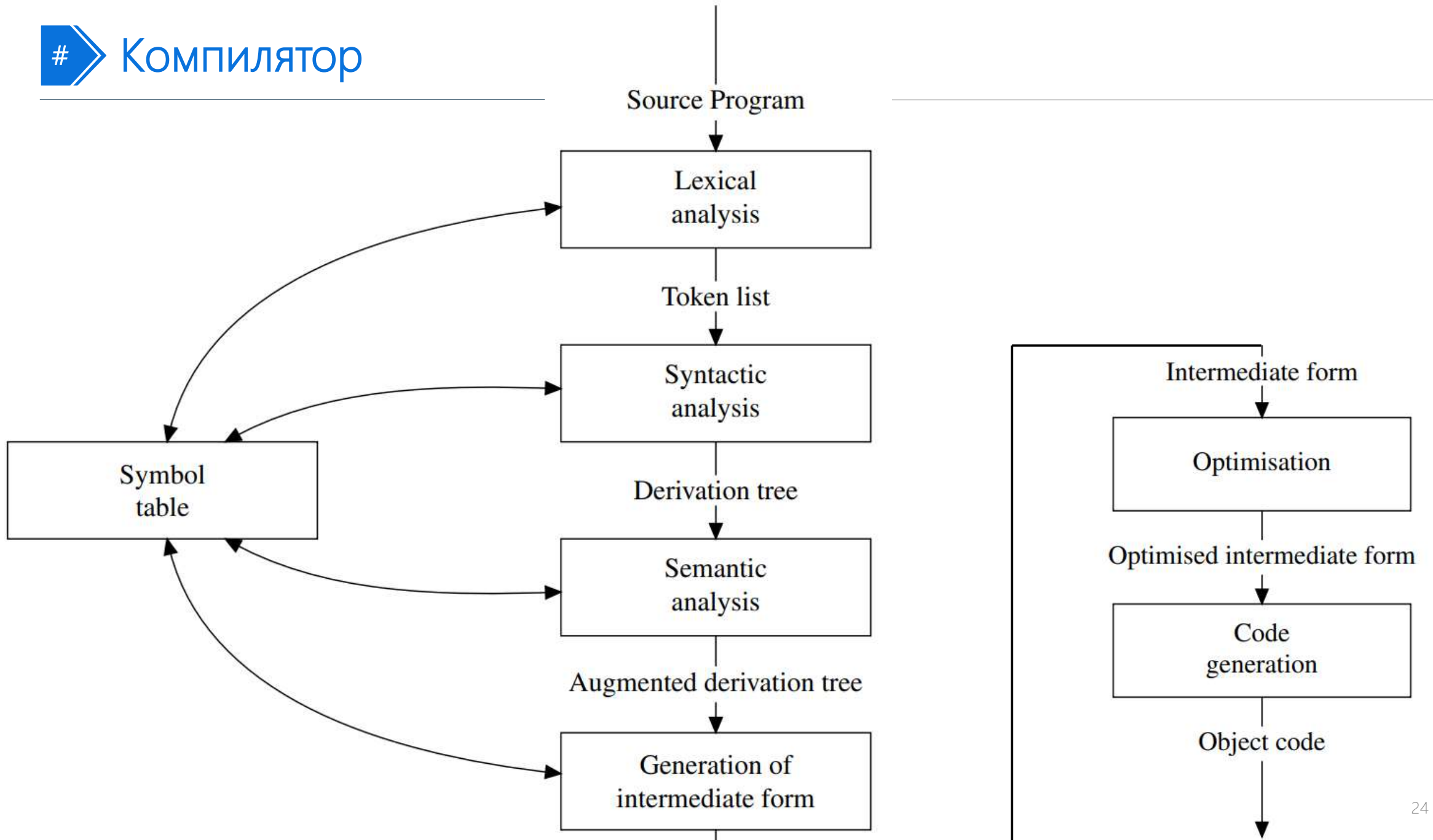
Контекст хамаарсан Синтакс шаардлага

- Contextual Syntactic Constraints
 - Танигчийг хэрэглэхээсээ өмнө зарласан байх ёстой (Pascal, C, Java).
 - Функцийн параметруудийн тоо нь формал параметруудтэй ижил байх ёстой (C, Pascal, Java гэх мэт).
 - Илэрхийллийн төрөл нь хувьсагчтай тохирох ёстой (C, Pascal, Java гэх мэт).
 - For давталтын хяналтын хувьсагчийг засварлахыг зөвшөөрдөггүй (Паскаль).
 - Хувьсагчийг хэрэглэхийн өмнө түүнд утга оноосон байх ёстой (Java, Python).
- Синтакс хязгаарлалт боловч контекстоос хамаардаг
 - CFG-ээр дүрслэх боломгүй, Контекстоос хамаарсан хэлзүйн төрлүүд хэрэгтэй
 - Бичих, боловсруулахад бэрхшээлтэй.
 - Хөрвүүлэгчид зориулсан үр ашигтай, CFG шиг автомат техник байхгүй.
 - Синтаксийн контекст бус дүрслэл шаардлагатай
 - Байгалийн хэл, Формал техник (шилжилтийн/transition систем) гм хэрэгтэй

- Программчлалын хэлний нэр томьёонд
 - *Синтакс*: Ихэвчлэн контекст-хамааралгүй дүрмээр дүрслэх боломжтой
 - *Статик семантик*: статик үндэслэл дээр программын текст ашиглан шалгах боломжтой контекст хамааралтай шаардлагаар дүрслэх боломжтой.
 - *Динамик семантик*: программ биелэгдэх үеийн бүх зүйлийг хэлдэг.
- Контекст-хамааралгүй ба контекст-хамааралтай синтакс
 - Ялгаа нь контекст-хамааралгүй хэлзүйн хүчин чадлаас хамаарна.
 - Синтакс ба семантикийн ялгаа нь үргэлж тодорхой байдаггүй.

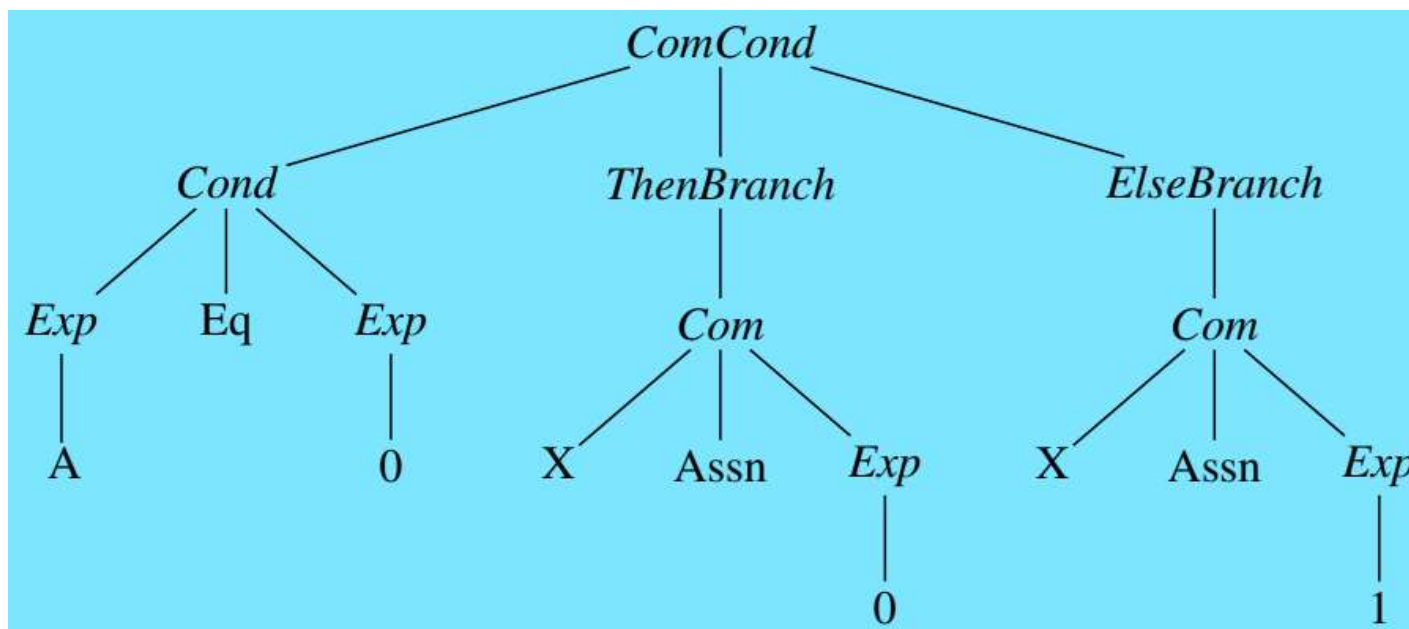
```
int A, B;  
read(A);  
B = 10/A;
```

0-д хуваагдах боломжтой програм



- Программын тэмдэглэгээнүүдийг (тэмдэгтүүдийг) оролтоос дараалан уншиж, токен буюу утга агуулга бүхий логик нэгж болгон бүлэглэнэ
 - $x = 1 + \text{foo} + +;$ (7 токен болно. C/Java lexical analyzer)
 - Чухал токенууд: хэлний тусгай үгс (for, if, else г.м), операторууд, нээх ба хаах хаалтууд (C/Java хэл дээрх { ба }, Python хэлний догол мөр г.м).
- Лексик шинжилгээ (эсвэл сканнердах)
 - Оролт дээр зүүн-баруун сканыг нэг удаа хийж токенуудыг танина.
 - Жишээнд токен дарааллыг хараахан шалгаагүй (хаалтны тэнцвэр).
 - Үндсэн хэрэгсэл нь үүсгэгч хэлзүйн тусгай анги (регуляр хэлзүй).
- Тайлбар толины (lexicon) элементийг дүрслэхэд
 - Элементийг үр дүнтэй олж мэдрэх хэрэгцээ
 - PL-ий тайлбар толь хязгааргүй тооны элементтэй байж болно
 - Ердийн толь бичгийн шиг энгийн жагсаалт хангалтгүй

- Токен жагсаалтад зориулсан гаргалгааны модыг бүтээхийг оролддог.
 - Навч бүр нь токен, зүүн-баруун уншихад зөв хэллэг (эсвэл терминал) үүсгнэ.



Хэлзүйн
гаргалгааны
мод

- Гаргалгааны модыг бүтээх боломжгүй байж болно.
 - Оролтын тэмдэг мөр хэлзүйн үүднээс буруу байх.
 - Энэ тохиолдолд задлан шинжлэгч алдааг мэдээлж, компиляцыг зогсооно.

- Гаралгааны мод (оролтын тэмдэгт мөрийн зөв синтакстай болохыг төлөөлдөг) нь хэлний янз бүрийн контекст дээр суурилсан шаардлагыг шалгадаг.
- Бидний харж байгаагаар энэ хэсэгт зарлагаа, төрөл, функцийн параметрийн тоо гэх мэтийг боловсруулдаг.
- Эдгээр шалгалтыг хийснээр гаргалгааны модыг тэдгээрээс олж авсан мэдээллээр **баяжуулж**, шинэ бүтэц бий болдог.
- Жишээлбэл, хувьсагчийн танигчийн токен бүр нь төрөл, зарласан байрлал болон бусад хэрэгтэй мэдээлэлтэй холбогдсон байна
- Ашиггүй давхардлаас зайлсхийхийн тулд энэ мэдээллийг ерөнхийдөө гаргалгааны модны гадна тусдаа бүтцэд цуглуулдаг.
- Эдгээр бүтцийн дотроос танигчийн талаарх мэдээллийг цуглуулдаг бүтцийг тэмдэгээний хүснэгт (symbol table) гэж нэрлэдэг.
- Тэмдэглэгээний хүснэгт нь залгамжлагч үе шатуудад чухал үүрэг гүйцэтгэдэг.

- Өргөтгөсөн гаргалгааны модны зөв нэвтрэлт нь код үүсгэх эхлэлийн шатыг боломжтой болгодог. Олон тооны оновчлол хийх үлдсэн бөгөөд тэдгээр нь тусгай объект хэлээс салангид байдаг учраас объектын хэлээр код үүсгэх хараахан боломжгүй байна. Түүгээр ч зогсохгүй компиляторыг зөвхөн нэг биш харин бүтэн объект хэлний код (жишээлбэл, өөр өөр архитектурт зориулсан машины од) үүсгэхийн тулд ерөнхийдөө хэрэгжүүлдэг. Тиймээс тодорхой хэлтэй холбоотой бүх сонголтыг нэг үе шатанд төвлөрүүлж, эх хэл болон объектын хэлнээс хамааралгүй байхаар зохион бүтээсэн завсрын хэлбэрээр код үүсгэх нь ашигтай.

- Өмнөх үе шатуудаас гаргалгааны модыг дахин дахин нэвтрэх замаар олж авсан код нь нэлээд үр ашиггүй байна. Объект код үүсгэхээс өмнө олон тооны оновчлолуудыг хийж болно. Гүйцэтгэх боломжтой ердийн үйлдлүүд нь:
 - Үхсэн кодыг цэвэрлэгээ. Өөрөөр хэлбэл, тэдгээрийн дуудаж болох гүйцэтгэлийн дараалал байхгүйгээс хэзээ ч гүйцэтгэх боломжгүй кодын хэсгүүдийг устгах.
 - Функцийн дуудалтын инлайн өргөтгөл. Зарим функцийн (процедурын) дуудалтыг харгалзах функцийн их биеээр орлуулж, гүйцэтгэлийг хурдан болгодог. Энэ нь бусад оновчлолыг хийх боломжтой болгодог.
 - Дэд илэрхийллийн задлал. Зарим программууд ижил утгыг нэгээс олон удаа тооцдог. Хэрэв энэ баримтыг компилятор илрүүлсэн бол нийтлэг дэд илэрхийллийн утгыг зөвхөн нэг удаа тооцоолж, хадгалсан байх боломжтой.
 - Давталтын оновчлол. Давталт нь хамгийн том оновчлол хийх боломжтой газар. Эдгээрийн дотроос хамгийн түгээмэл нь төрөл бүрийн давталтын дотор утга тогтмол хэвээр байгаа дэд илэрхийллүүдийн тооцооллыг давталтан дотроос хасах явдал юм.

- Оновчилсон завсрын хэлбэрээс эцсийн объектын кодыг үүсгэнэ. Дараа нь ерөнхийдөө оновчлолын тухайн объект хэлний онцлогт суурилсан хамаарсан сүүлчийн үе шат байдаг. Машины кодыг үүсгэдэг компиляторын хувьд энэ сүүлчийн үе шатны чухал хэсэг нь регистрийн хуваарилалт (ямар хувьсагчийг ямар процессорын регистрт хадгалах тухай шийдвэр). Энэ нь эцсийн программын үр ашгийг дээшлүүлэх зориулсан асар чухал сонголт юм.



ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл, Холбооны Технологийн Сургууль

АНХААРАЛ ТАВЬСАНД БАЯРЛАЛАА